Expense Mate
Members: Areesha 24k-3043, Burhan 24k-3056
Phase 02
Architecture & Construction Planning

1. Overview & Objectives

In Phase 2 of the Software Construction lifecycle, the baseline specifications defined in Phase 1 (SRS) are transformed into an actionable, verified architectural blueprint. At this stage, requirements (FR-1 to FR-16 and NFR-1 to NFR-15) are mapped to concrete module boundaries and operational guidelines before coding starts in Phase 3. This report formalizes architectural style selection, four complementary structural views, coding standards, development tooling, and the construction schedule.

2. Architectural Style Evaluation & Selection

Selecting an architectural style establishes how components communicate and maintain data integrity. Three candidates were evaluated against project constraints:

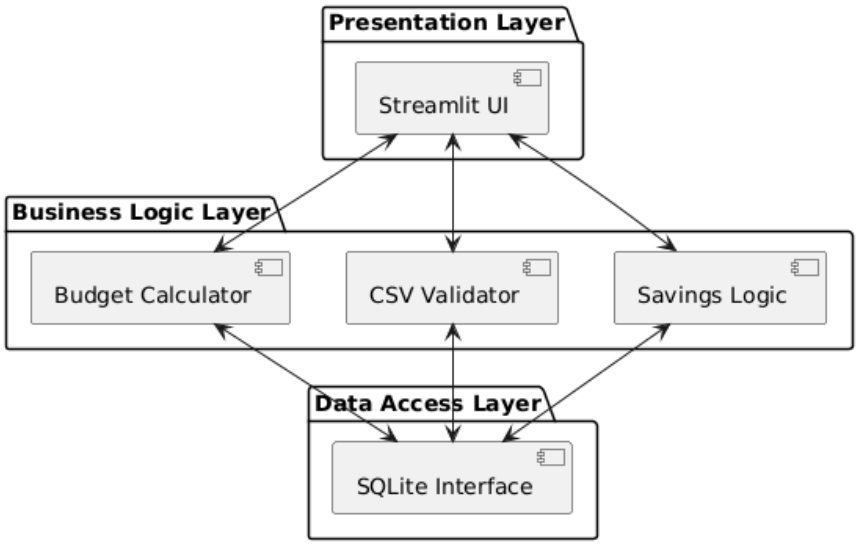
Architectural Style	Structural Description	Key Advantages	Limitations for ExpenseMate	Decision
Layered (3-Tier)	Presentation, Business Logic, and Data Persistence separated into strict tiers.	Clean separation of concerns, high testability, zero network latency, minimal boilerplate.	Requires discipline to prevent layer leaking: UI must never invoke SQL directly.	Selected
Client-Server (REST)	Frontend client communicating with a remote web API service over JSON.	Independent frontend and backend deployment: supports multi-client ecosystems.	High network serialization overhead and operational complexity for single-user local tool.	Rejected
Peer-to-Peer	Decentralized mesh where nodes communicate and synchronize state directly.	High fault tolerance with no single point of failure in distributed multi-user environments.	Severe distributed state synchronization overhead, irrelevant for standalone desktop finances.	Rejected

Selection Rationale: The Three-Tier Layered Architecture is selected unanimously by both members. It satisfies NFR-7 by isolating the UI from database logic, guarantees sub-second responsiveness (NFR-1, NFR-13), runs offline on student machines (NFR-14), and enables thorough unit testing (NFR-8).

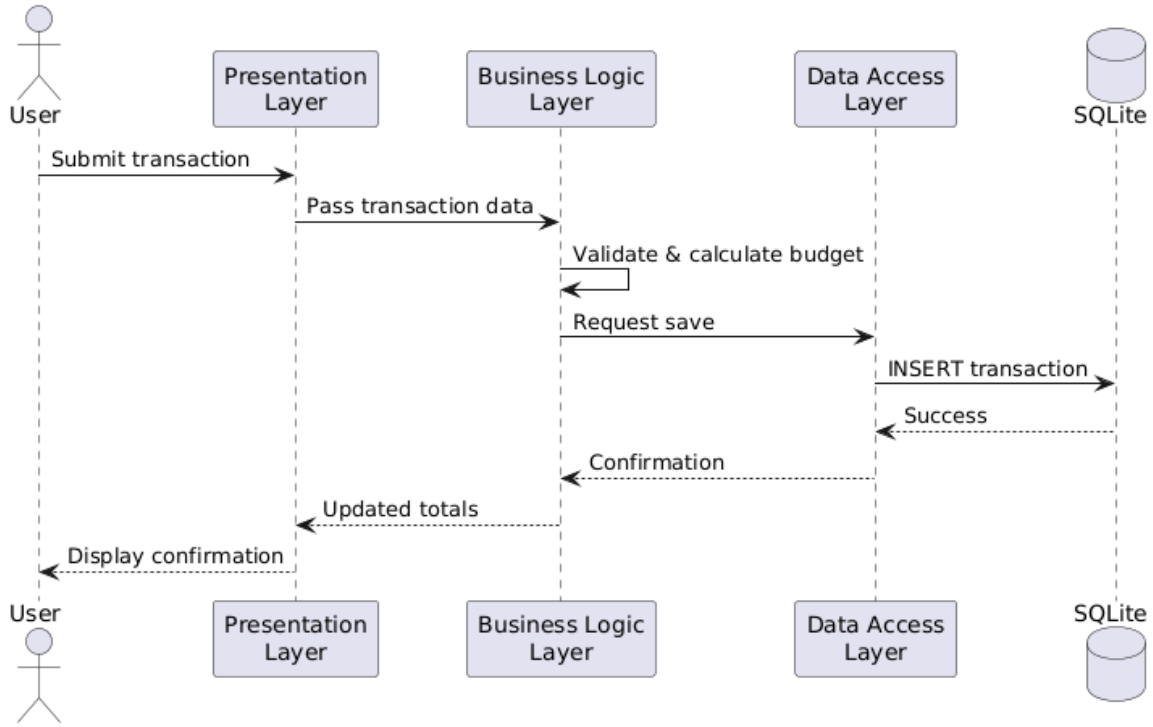
3. High-Level Architectural Views

The system structure is formalized across four architectural views that govern construction:

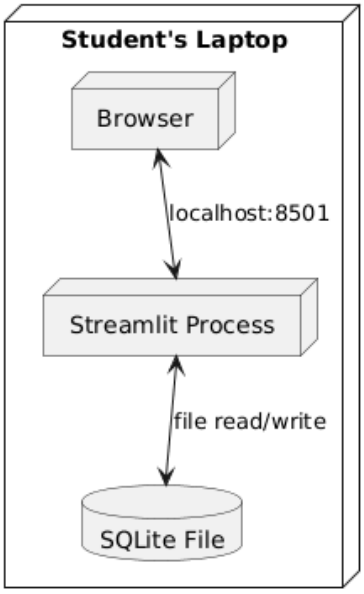
3.1 Logical View



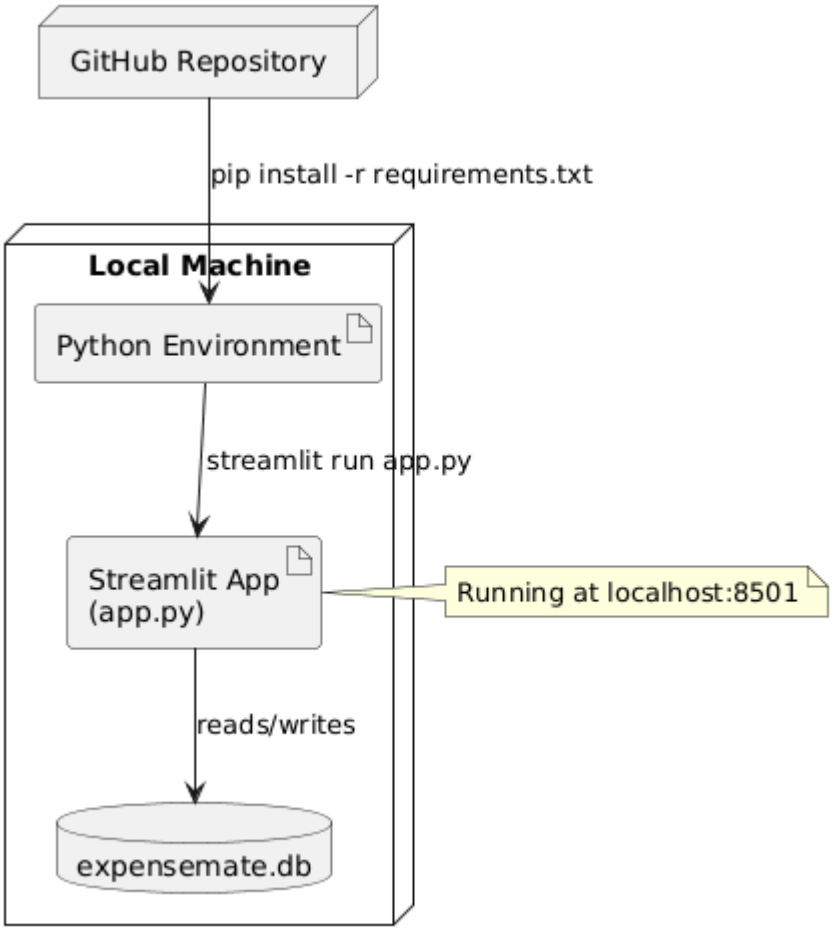
3.2 Process View



3.3 Physical View



3.4 Deployment View



4. Coding Standards

Strict conventions ensure architectural integrity, maintainability, and seamless pair programming during Phase 3 construction:

Naming & Formatting: Strict adherence to PEP 8. Functions and variables use snake_case (e.g. calculate_spent), classes use PascalCase (e.g. ValidationError), constants use UPPER_SNAKE_CASE (e.g. DEFAULT_CATEGORIES), and module filenames use lowercase with underscores.

Layer Isolation: Presentation code must never import sqlite3 or execute SQL statements. Service modules must never import streamlit or render UI widgets. Data repositories must return plain Python dictionaries or primitive types to keep layers completely decoupled (NFR-7).

Defensive Data Handling: Monetary values are parsed and manipulated in integer minor units (cents) to avoid floating-point inaccuracies. Dates require ISO YYYY-MM-DD calendar validation between 1900 and 2100 (NFR-11). Validation exceptions are raised early to prevent malformed data from reaching SQLite (NFR-5, NFR-9).

5. Tooling Stack & Technology Decisions

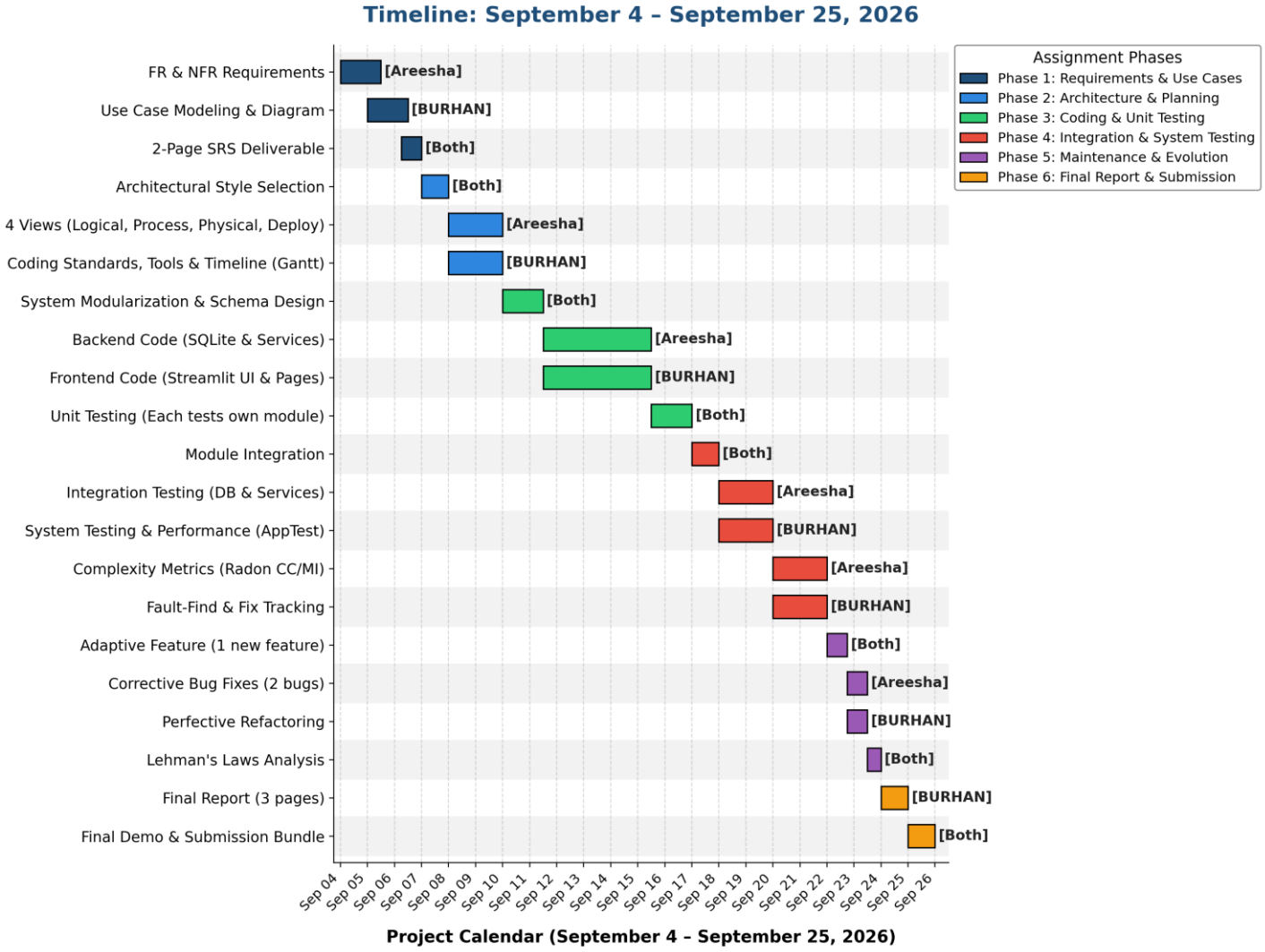
Selected tools are optimized for high developer productivity, robust automated testing, and zero external infrastructure setup:

Category	Selected Technology	Architectural Role & Justification
Language & Runtime	Python 3.12+	Core execution platform providing rapid prototyping, built-in sqlite3 support, and modern type annotations.
Frontend UI	Streamlit (>= 1.64)	Declarative Python-native UI engine enabling responsive dashboards without JavaScript complexity.
Data Processing	Pandas (>= 2.2)	Efficient tabular data structures for transaction sorting, filtering, and seamless CSV export and import.
Data Visualization	Matplotlib (>= 3.8)	Programmatic rendering engine generating multi-series budget and savings trend charts.
Persistence Engine	SQLite 3 (Embedded)	Serverless, zero-configuration ACID relational database storing data in a single portable file (expensemate.db).
Test Automation	Pytest & Pytest-cov	Automated test execution harness providing parameterized testing and statement/branch coverage metrics (NFR-8).
Headless UI Testing	Streamlit AppTest API	Automated testing tool simulating user form submissions, button interactions, and session state verification.
Code Quality	Radon	Static analysis profiler measuring Cyclomatic Complexity and Maintainability Index scores.
Version Control	Git & GitHub	Distributed revision control for task tracking, branch merging, and collaborative peer reviews.

6. Construction Timeline & Gantt Schedule

The project schedule spans 22 days (September 4 to September 25, 2026). Durations are rationally weighted: requirements and planning establish solid foundations (3 days each), coding is allocated the largest block (7 days), and verification and maintenance occur systematically.

Lifecycle Phase Breakdown: Phase 1: Requirements (3 days) | Phase 2: Architecture & Planning (3 days) | Phase 3: Coding & Unit Testing (7 days) | Phase 4: Integration & System Testing (5 days) | Phase 5: Maintenance & Evolution (2 days) | Phase 6: Final Submission (2 days).



7. Requirements Traceability to Architecture

Every architectural component maps directly to requirements ratified in Phase 1 (SRS):

Phase 1 Requirement	Architectural Realization	Module Allocation
FR-1 to FR-4 (Transactions)	Multi-field input forms, dynamic filters, and CRUD persistence pipelines.	pages/1_Transactions.py, services/transaction_service.py, data/transaction_repo.py
FR-5, FR-6 (Categories)	Category repository with foreign key validation preventing orphan records.	pages/2_Categories.py, services/category_service.py, data/category_repo.py
FR-7 to FR-9 (Budgets)	Standing monthly budget storage, spent calculations, and alert indicators.	pages/3_Budgets.py, services/budget_service.py, services/budget_calculator.py
FR-10, FR-11 (Portability)	CSV parser with atomic rollback ensuring all-or-nothing batch insertion.	pages/5_Import_Export.py, services/csv_validator.py, data/database.py
FR-12 to FR-15 (Analytics)	Matplotlib rendering routines for budget distributions and savings trends.	pages/4_Analytics.py, services/savings_logic.py, ui/charts.py
FR-16, NFR-4 (Persistence)	Embedded SQLite database enforcing constraints on every connection.	data/database.py, expensemate.db
NFR-1, NFR-13 (Performance)	Direct indexed SQL queries executing within milliseconds (<2.0s limit).	data/transaction_repo.py, data/budget_repo.py
NFR-5, NFR-9, NFR-11 (Integrity)	Fail-fast validation rejecting negative numbers, zero limits, and bad dates.	services/validators.py, services/money.py
NFR-6 (CSV Integrity)	Transactional boundaries guaranteeing all-or-nothing batch file ingestion.	services/csv_validator.py, data/database.py
NFR-7 (Architecture)	Three-Tier layered structure separating UI, services, and database logic.	pages/, services/, data/
NFR-8 (Testing)	Decoupled architecture enabling automated unit and headless testing.	tests/ test suite using Pytest and Streamlit AppTest
NFR-14 (Environment)	Self-contained local execution on standard student computers with zero servers.	Local Python runtime and SQLite engine

8. Phase 2 Conclusion & Construction Readiness

Phase 2 successfully translates Phase 1 requirements into a clear, modular, and testable design. Architectural style evaluation, four complementary views, coding standards, tooling decisions, and the construction timeline have been fully ratified. With service boundaries established and quality gates configured, the system is primed for Phase 3 construction.